

Universidad Tecnológica Nacional

Tecnicatura en Programación a Distancia

Programación 1

Trabajo Práctico Integrador

Gestión de Datos de Países en Python: filtros, ordenamientos y estadísticas

Integrantes	
Nombre	Apellido
Thiago	Lugo
Lautaro	Torres

Fecha de Entrega: 15/06/2026

1. Marco Teórico2

1.1 Listas en Python	3
1.2 Diccionarios en Python.....	3
1.3 Funciones	3
1.4 Estructuras Condicionales y Repetitivas	4
1.5 Ordenamientos.....	4
1.6 Estadísticas básicas.....	4
1.7 Archivos CSV.....	5
2. Decisiones Técnicas y Arquitectura	5
2.1 Estructura del código	5
2.2 Diagrama de flujo del programa.....	5
2.3 Decisiones de diseño	6
3. Dificultades y Conclusiones	6
4. Bibliografía/Webgrafía	7
5. Links del proyecto	8

1. Marco Teórico

1.1 Listas en Python

Una lista es una estructura de datos ordenada y mutable que permite almacenar múltiples elementos de distintos tipos bajo un mismo nombre de variable. En Python se definen con corchetes y sus elementos se separan por comas. Las listas admiten duplicados, pueden modificarse en tiempo de ejecución y ofrecen métodos integrados como `append()`, `remove()`, `sort()` y `len()`, entre otros.

En este proyecto, la lista es la estructura principal de almacenamiento: cada país se representa como un diccionario, y todos los países se agrupan en una lista llamada `países`. Esto permite recorrer el dataset completo con bucles y aplicar filtros y ordenamientos de forma eficiente.

Ejemplo de uso en el proyecto:

```
países = []          # Lista vacía inicial
países.append(nuevo_pais) # Agregar un país
len(países)         # Cantidad de países
```

1.2 Diccionarios en Python

Un diccionario es una estructura de datos que almacena pares clave-valor. Cada clave es única dentro del diccionario y permite acceder directamente al valor asociado sin necesidad de recorrer toda la estructura. Se definen con llaves `{}` y se accede a sus elementos mediante la clave entre corchetes.

En este proyecto, cada país se representa como un diccionario con cuatro claves fijas: `nombre`, `poblacion`, `superficie` y `continente`. Esta elección permite acceder a cualquier dato del país de forma directa y legible, haciendo el código más claro y mantenible.

```
pais = {
    "nombre": "Argentina",
    "poblacion": 45376763,
    "superficie": 2780400,
    "continente": "América"
}
```

1.3 Funciones

Una función es un bloque de código reutilizable que realiza una tarea específica y puede recibir parámetros de entrada y retornar un resultado. El uso de funciones permite

modularizar el código, facilitando su lectura, mantenimiento y prueba. En Python se definen con la palabra clave `def`.

El principio aplicado en este proyecto es 'una función = una responsabilidad': cada función tiene un único propósito bien definido. Por ejemplo, `cargar_paises()` solo lee el CSV, `filtrar_por_continente()` solo filtra, y `mostrar_tabla()` solo muestra resultados. Esto hace que el código sea fácil de depurar y ampliar.

1.4 Estructuras Condicionales y Repetitivas

Las estructuras condicionales (`if / elif / else`) permiten tomar decisiones en el flujo del programa según el valor de una condición booleana. Las estructuras repetitivas (`for` y `while`) permiten ejecutar un bloque de código múltiples veces.

En este proyecto se utilizan en múltiples contextos: el bucle `while True` mantiene el menú activo hasta que el usuario elige salir; los bucles `for` recorren la lista de países para filtrar, buscar o mostrar datos; y los condicionales validan entradas del usuario y controlan el flujo según la opción elegida.

1.5 Ordenamientos

El ordenamiento es el proceso de reorganizar una colección de datos según un criterio determinado. Python provee la función integrada `sorted()` que retorna una nueva lista ordenada sin modificar la original, y acepta un parámetro `key` para especificar el criterio de comparación, y `reverse` para invertir el orden.

En el sistema se utiliza `sorted()` con funciones `lambda` como clave. Por ejemplo, para ordenar por nombre de forma ascendente:

```
resultado = sorted(paises, key=lambda p: p['nombre'].lower(), reverse=False)
```

El uso de `.lower()` garantiza que el ordenamiento alfabético no distinga entre mayúsculas y minúsculas.

1.6 Estadísticas básicas

Las estadísticas básicas permiten resumir y caracterizar un conjunto de datos numéricos. En este proyecto se calculan: el máximo y mínimo de población usando `max()` y `min()` con clave

lambda; el promedio de población y superficie usando `sum()` dividido `len()`; y la distribución de países por continente mediante un diccionario de conteo.

1.7 Archivos CSV

CSV (Comma-Separated Values) es un formato de texto plano para almacenar datos tabulares. Cada fila representa un registro y los valores de cada columna se separan por comas. Python incluye el módulo `csv` en su biblioteca estándar, que provee las clases `DictReader` y `DictWriter` para leer y escribir CSV usando diccionarios, lo que hace el código más legible al acceder a los campos por nombre en lugar de por índice.

2. Decisiones Técnicas y Arquitectura

2.1 Estructura del código

El código fue organizado en 6 módulos lógicos dentro de un único archivo Python, cada uno con una responsabilidad bien definida:

Módulo	Funciones Principales	Responsabilidad
1 - Archivos	<code>cargar_paises()</code> , <code>guardar_paises()</code>	Lectura y escritura del CSV
2 - Alta/Baja	<code>agregar_pais()</code> , <code>actualizar_pais()</code>	Gestión de datos
3 - Búsqueda	<code>buscar_pais()</code> , <code>filtrar_por_*</code> ()	Búsqueda y filtros
4 - Orden	<code>ordenar_paises()</code>	Ordenamiento ascendente/descendente
5 - Estadísticas	<code>mostrar_estadisticas()</code>	Cálculo de indicadores
6 - Utilidades	<code>mostrar_tabla()</code> , <code>pedir_*</code> ()	Helpers y validaciones

2.2 Diagrama de flujo del programa

El siguiente diagrama representa el flujo general de ejecución del sistema:

INICIO
Cargar datos desde paises.csv
Mostrar menú principal
Leer opción del usuario
Opción 1: Agregar país → Validar campos → Guardar en CSV
Opción 2: Actualizar país → Buscar exacto → Modificar → Guardar
Opción 3: Buscar por nombre → Búsqueda parcial → Mostrar resultados
Opción 4/5/6: Filtrar → Aplicar condición → Mostrar resultados
Opción 7: Ordenar → Elegir campo y orden → Mostrar tabla ordenada
Opción 8: Estadísticas → Calcular indicadores → Mostrar
Opción 9: Mostrar todos los países
Opción 0: FIN

2.3 Decisiones de diseño

Persistencia en CSV: Se eligió el formato CSV por su simplicidad, compatibilidad con la consigna del trabajo y por ser suficiente para el volumen de datos del dataset. Cada modificación (agregar o actualizar un país) se guarda inmediatamente en el archivo.

Validaciones: Todas las entradas del usuario son validadas antes de procesarse. Los campos numéricos se convierten con `int()` dentro de bloques `try/except` para capturar errores de formato. Los campos de texto se verifican con `.strip()` para evitar espacios en blanco como valores válidos.

Menú con diccionario de funciones: El menú principal utiliza un diccionario que mapea cada opción numérica a una función lambda. Esto evita largas cadenas de `if/elif` y hace el código más limpio y extensible.

3. Dificultades y Conclusiones

Dificultades encontradas

Durante el desarrollo del trabajo nos encontramos con distintos obstáculos técnicos que nos obligaron a investigar y profundizar en los conceptos de la materia:

Lectura del CSV con caracteres especiales: Al trabajar con nombres de países que contienen tildes y caracteres especiales (como Japón, Argelia, etc.), nos encontramos con errores de codificación. La solución fue especificar `encoding='utf-8'` explícitamente en la apertura del archivo.

Validación de entradas del usuario: Garantizar que el programa no falle ante entradas inválidas (letras donde se esperan números, campos vacíos, rangos invertidos) requirió diseñar funciones auxiliares de validación reutilizables como `pedir_entero_positivo()`.

Persistencia de datos: Asegurarse de que los cambios realizados en memoria se reflejaran correctamente en el archivo CSV implicó llamar a `guardar_paises()` después de cada modificación y manejar posibles errores de escritura.

Conclusiones

Este trabajo nos permitió consolidar los conceptos fundamentales de Programación 1 aplicándolos a un problema concreto y completo. La modularización del código en funciones con responsabilidades únicas resultó clave para mantener el proyecto organizado y fácil de depurar.

El uso combinado de listas y diccionarios demostró ser una solución elegante para representar datos estructurados sin necesidad de librerías externas. Aprendimos que anticipar los posibles errores del usuario y manejarlos con mensajes claros es tan importante como la funcionalidad principal del programa.

En términos de trabajo en equipo, dividir las funcionalidades por módulos nos permitió trabajar en paralelo y luego integrar el código de forma ordenada.

4. Bibliografía/Webgrafía

[1] Python Software Foundation. Documentación oficial de Python 3. Recuperado de: <https://docs.python.org/3/>

[2] Python Software Foundation. Módulo csv — Lectura y escritura de archivos CSV.

Recuperado de: <https://docs.python.org/3/library/csv.html>

[3] Python Software Foundation. Funciones integradas: sorted(), max(), min(), sum().

Recuperado de: <https://docs.python.org/3/library/functions.html>

[4] Real Python. Reading and Writing CSV Files in Python. Recuperado de:

<https://realpython.com/python-csv/>

[5] Real Python. Python Dictionaries. Recuperado de: <https://realpython.com/python-dicts/>

[6] Real Python. How to Use Python Lambda Functions. Recuperado de:

<https://realpython.com/python-lambda/>

5. Links del proyecto

Repositorio GitHub: [Repositori GitHub](#)

Video demostrativo: [Video explicativo de Lautaro Torres y Thiago Lugo](#)